

Adaptive Scanning Simulation

Current Setup and Changes from the Initial `dev` Branch

Document status: Engineering summary
Compared branches: `dependency_fix` against the original `dev` branches for Common, Robot, and Sensor; against `main` for the two newly introduced repositories
Validation basis: Source comparison and an ArmX-E-triggered Gazebo scan run

1. Purpose

This document explains the current split-repository Adaptive Scanning setup and summarizes how it differs from the initial `dev` branch arrangement. The change separates application orchestration, robot motion, camera support, shared ROS interfaces, and simulation tooling into clear package boundaries.

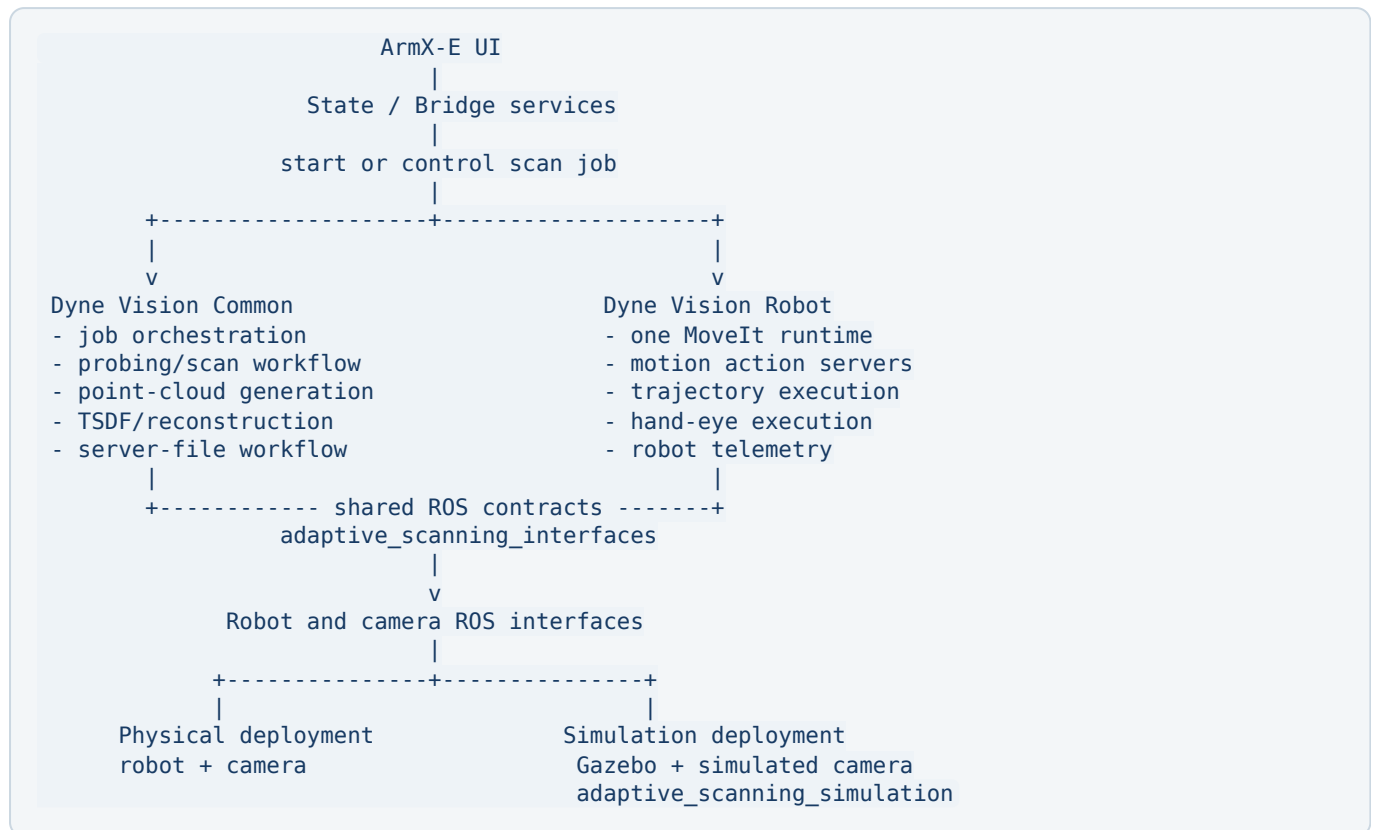
The current setup was validated by starting a scan job from ArmX-E. The simulated robot reached its scan pose, completed six probing waypoints, and completed six initial camera look-at targets. The later TSDF depth-range mismatch observed during that run is a separate simulation-parameter issue and is outside the scope of this document.

2. Current Repository Structure

Repository	Current responsibility	Comparison baseline
<code>Dyne-vision-common</code>	Scan orchestration, reconstruction/perception pipeline, ArmX-E services and UI bridge, application-level telemetry	<code>dev</code>
<code>Dyne-vision-robot</code>	Robot description integration, controllers, MoveIt, motion action servers, hand-eye execution, generic robot telemetry	<code>dev</code>
<code>Dyne-vision-sensor</code>	Camera-specific runtime and sensor support	<code>dev</code>
<code>adaptive_scanning_interfaces</code>	Public ROS message, service, and action contracts shared across containers	<code>main</code> (repository introduced for the split)
<code>adaptive_scanning_simulation</code>	Gazebo world, simulated robot/camera setup, test objects, simulation runner, readiness checks, and simulation documentation	<code>main</code> (repository introduced for the split)

All repositories currently use the branch name `dependency_fix` for this coordinated version.

3. Current Runtime Architecture



Common requests robot operations through ROS interfaces. Robot owns the implementation of those operations and the single MoveIt instance. This prevents Common from compiling against or embedding Robot motion implementation details.

4. Shared ROS Interface Package

The new `adaptive_scanning_interfaces` repository contains contracts only. It does not contain planning or execution logic.

The shared definitions currently include:

- `ExecuteJoint.action`
- `ExecuteWaypoints.action`
- `LookAtWaypoints.action`
- `ParallelPlanningAndExecution.action`
- `RunHandeyeCalibration.action`
- `ExecuteJoint.srv`
- `RobotHardwareStatus.msg`
- `WaypointPlannerStage.msg`

Both Common and Robot build against this package. Common acts as a client for robot-owned capabilities, while Robot provides the action or service implementations.

5. Current Container and Process Ownership

Capability	Initial dev ownership	Current ownership
Scan workflow and reconstruction	Common	Common
ArmX-E job bridge and application services	Common	Common
Movelit runtime	Common	Robot
Robot motion planning and execution servers	Common	Robot
Look-at planning and waypoint execution	Common	Robot
Hand-eye calibration execution	Common	Robot
Hand-eye request entry point	Common implementation	Lightweight Common client calling Robot action
Robot hardware telemetry	Common, including robot-specific dependency	Robot, published through a generic shared message
Cross-container ROS definitions	Defined inside Common	Dedicated interfaces repository
Gazebo package and simulation assets	Common repository	Dedicated simulation repository
Camera pipeline wrapper depending on Common	Sensor	Removed; Sensor remains camera-specific

6. Changes by Repository

6.1 Dyne Vision Common

Common was reduced to application-level responsibilities.

Key changes:

- Removed the locally defined robot motion actions, services, and messages; imports now use `adaptive_scanning_interfaces`.
- Removed `as_moveit_node` and the associated planning, look-at, roadmap, trajectory, dither, and parallel-planning implementations.
- Removed full hand-eye calibration execution code and configuration from Common.
- Added a lightweight hand-eye calibration client. It forwards the ArmX-E request to the Robot-owned `RunHandeyeCalibration` action and returns the result.
- Removed the robot hardware publisher from Common.
- Added `common_runtime.launch.py` for Common-owned runtime services.
- Updated scan, table-calibration, and orchestration nodes to consume external Robot interfaces.
- Updated Docker build dependencies to clone/build the shared interface package instead of compiling Robot functionality in Common.

- Removed the embedded `as_sim` package, simulation assets, runners, generated runtime artifacts, and simulation documentation after moving them to the dedicated simulation repository.
- Updated ArmX-E simulation and compose scripts to locate the simulation repository and share simulation runtime artifacts with the Common container.

Result: Common can run its scan workflow without owning MoveIt or robot-specific implementation code.

6.2 Dyne Vision Robot

Robot became the owner of all robot-side motion capabilities.

Key changes:

- Removed the direct package dependency on `as_edge_common`.
- Added the Adaptive Scanning MoveIt node and its supporting motion-planning implementation.
- Added Robot-owned action servers for joint execution, waypoint execution, look-at motion, and parallel planning/execution.
- Added motion parameters and integrated these processes into `robot.launch.py` and the Robot container entry point.
- Moved hand-eye calibration runtime, controllers, player, configuration, and helper modules into Robot.
- Added `RunHandeyeCalibration` action server for the Common/ArmX-E request boundary.
- Added a generic robot hardware publisher that reports through `RobotHardwareStatus` rather than exposing a Doosan-specific contract to Common.
- Retained the local Open3D compatibility helper in Robot for the current implementation.
- Removed the older duplicated Doosan MoveIt launch override so that the package has one owned launch path.

Result: one Robot container owns one MoveIt runtime used by normal scanning and hand-eye calibration.

6.3 Dyne Vision Sensor

The unused `camera_pipeline.launch.py` wrapper was removed because it imported Common-owned nodes and created an unnecessary Sensor-to-Common dependency.

Result: Sensor remains responsible for camera-specific functionality and no longer carries a launch-time dependency on Common.

6.4 Adaptive Scanning Interfaces

This repository was introduced as a small, public ROS interface package.

Result: Common and Robot share stable wire-level contracts without either container importing the other container's source package.

6.5 Adaptive Scanning Simulation

Simulation was extracted from Common into a dedicated development and test repository.

It now owns:

- the `as_sim` ROS package;
- Gazebo world and spawn launch files;
- simulated depth-camera bridge and frame relay;
- STL test-object dataset and simulation models;

- table-cloud generation;
- clock, sensor, controller, and action readiness checks;
- staged commands for build, prepare, Gazebo, MoveIt, scan, hardware-only mode, status, logs, and cleanup;
- instructions for standalone and ArmX-E-triggered simulation.

Result: production Common, Robot, and Sensor repositories do not need to own Gazebo assets or test datasets.

7. Simulation Execution Paths

7.1 Standalone ROS Simulation

The simulation runner starts the complete ROS scan path without ArmX-E:

1. Build the required packages in a shared ROS 2 workspace.
2. Generate simulation parameters and table calibration data.
3. Start Gazebo with the robot, camera, controllers, and selected test object.
4. Start the Robot launch in Gazebo mode, including the single MoveIt instance and Robot action servers.
5. Start the Common scan-and-perceive workflow.
6. Monitor scan logs and ROS status through the simulation runner.

7.2 ArmX-E-Triggered Simulation

The application-integrated path is:

1. Start simulated hardware: Gazebo, camera topics, controllers, and Robot/MoveIt runtime.
2. Start ArmX-E services from Common against the same ROS domain.
3. Onboard or use the configured fleet connection.
4. Select a scan job in ArmX-E and confirm **Start**.
5. ArmX-E State Manager coordinates the job and requests the Bridge to launch Common's `edge_scan.launch.py`.
6. Common reads the job inputs and calls Robot-owned actions for scan-pose movement, probing, and camera-view motion.
7. Robot plans and executes trajectories through its single MoveIt runtime.
8. Common collects camera data and advances reconstruction and NBV stages.
9. Status and logs are returned through the ArmX-E application services.

8. Validated Behaviour

The current split was tested without changing repository source code. The test confirmed:

Validation point	Result
Gazebo robot, camera, table, and selected STL loaded	Passed
Simulation clock advanced monotonically after the world was unpaused	Passed
Camera depth and point-cloud topics published	Passed

Validation point	Result
Robot controllers became active	Passed
One MoveIt runtime was available in Robot	Passed
Robot motion actions were discoverable from Common	Passed
ArmX-E services and local UI started	Passed
Job selected and started through the ArmX-E UI	Passed
Common edge scan launched through the Bridge	Passed
Robot moved to configured scan pose	Passed
Six probing waypoints planned and executed	Passed
Six initial camera look-at targets planned and executed	Passed
Camera frames reached TSDF reconstruction	Passed
Completion beyond initial reconstruction	Not evaluated here; separate depth-range configuration issue

9. Current Operational Notes

- Every process participating in one simulation run must use the same `ROS_DOMAIN_ID`.
- Host execution needs the ROS 2 Jazzy Doosan dependencies used by Robot and the Python dependencies used by Common, including Open3D.
- Gazebo must be actively running, not merely publishing a discovered `/clock` topic. Readiness should verify multiple increasing clock samples.
- The ArmX-E simulation path also requires its PostgreSQL service.
- The Robot hardware telemetry helper currently has a diagnostic-level byte conversion issue in simulation. This does not prevent scan motion but should be corrected before relying on simulated robot telemetry.
- If a launched scan process exits unexpectedly, the persisted ArmX-E job state should be reconciled so the UI does not continue to show `Running` after the Bridge returns to `IDLE`.

10. Summary

The initial `dev` arrangement placed simulation, MoveIt, robot motion, hand-eye calibration, telemetry, and shared ROS definitions largely inside Common. The current setup establishes five explicit ownership areas:

1. Common owns application orchestration and scanning/reconstruction logic.
2. Robot owns motion, MoveIt, calibration execution, and robot telemetry.
3. Sensor owns camera-specific runtime functionality.
4. Interfaces owns the shared ROS contracts.
5. Simulation owns Gazebo and development-only test infrastructure.

This structure removes direct Common/Robot/Sensor source dependencies while preserving ROS-based runtime communication and the ArmX-E-controlled scan flow.